Software Architecture and Design Specification

Project: UATC – Universal Adaptive Transform Compression

Version: 1.0

Authors: Team UATC – S Chakshu, Sai Rishi Gangarapu (Team Lead), Sampada K, Manasa K

Date: 29-09-2026

Status: Draft

Project Guide: Dr. Sapna V M

Course: Level 3 Software Engineering Mini Project (UE24CS341A), PES University

Revision History

Version	Date	Author	Change Summary
0.1	22-09-2026	Team UATC	Outline created from SRS v1.0 and project synopsis
1.0	29-09-2026	Team UATC	First complete draft: architecture, design, API and security sections

Approvals

Role	Name	Signature/Date
Project Guide	Dr. Sapna V M	
Team Lead	Sai Rishi Gangarapu	
Team Member	S Chakshu	
Team Member	Sampada K	
Team Member	Manasa K	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of UATC (Universal Adaptive Transform Compression), a block-adaptive, lossless file compression and archival tool with a command line interface and a browser-based frontend. It turns the requirements in the UATC SRS v1.0 into concrete modules, interfaces, data formats and interaction flows, so that the team can implement in parallel and the testers can derive test cases from it.

1.2 Scope

Covers the UATC compression core (file analysis, compressibility probing, adaptive transform selection, entropy coding, integrity checking, parallel block execution), the versioned .uatc archive format, the API/application layer, the CLI, and the web frontend. Not covered: lossy compression, audio/video codecs, a native desktop application and distributed execution (all out of scope per SRS Section 1.2).

1.3 Audience

Project guide and evaluation panel, developers of the four UATC subsystems, QA engineers writing the Software Test Plan (STP), and anyone who maintains or extends the transform set later.

1.4 Definitions

Term	Meaning
UATC	Universal Adaptive Transform Compression; also the name of the archive format (.uatc)
Block	A bounded-size chunk of an input file that is analysed, transformed and coded independently
Transform	A reversible operation applied to a block before entropy coding (None, Delta, XOR, Byte-shuffle, RLE)
Entropy coder	The final lossless coding stage applied to the chosen block representation
Probe	Cheap estimate of whether a block is worth compressing at all
Store	Keeping a block's original bytes unchanged because compression would not make it smaller
CRC32 / SHA-256	Checksums used for block-level and archive/file-level integrity
ADR	Architecture Decision Record
API / ABI	Application Programming Interface / Application Binary Interface (here: the on-disk archive layout)
STRIDE	Threat model categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
SRS / STP / RTM	Software Requirements Specification / Software Test Plan / Requirements Traceability Matrix

2. Document Overview

2.1 How to use this document

Section 3 explains the architecture: goals, components, the chosen pattern, technology, risks, traceability and security. Section 4 gives the detailed design: module layout, the archive format, UML sequence diagrams, API and interface definitions, error handling and UX. Module owners should read Sections 3.4 and 4.3 for the interface they must implement. Testers should use Section 3.8 (traceability) and 4.3 (interfaces and error codes).

2.2 Related Documents

- UATC Software Requirements Specification, v1.0 (September 2026)
- UATC Project Synopsis – Web Frontend (Final)
- UATC Software Test Plan (STP), v1.0
- Requirements Traceability Matrix (SRS Appendix C, extended in the STP)

3. Architecture

3.1 Goals & Constraints

Goals

- Correctness first: every extracted file is byte-for-byte identical to the original (NFR Correctness).
- Corruption is always detected and reported, never silently turned into wrong output (FR-09, FR-12).
- Per-block adaptivity: each block uses the reversible transform with the lowest estimated cost, or is stored unchanged (FR-05 to FR-08).
- Deterministic output: the archive is identical whether built with 1 or N worker threads (FR-10).
- Modularity: a new transform can be added without changing the archive format, API, frontend or CLI.
- Two interfaces on one core: CLI for technical users, web UI for non-technical users, with no compression logic in the frontend.

Constraints

- Core written in C++17, built with CMake (SRS 2.5).
- Archive format is versioned from the first release.
- Candidate transform set for v1 is fixed and small; one experimental transform is a stretch goal only after Phase 4.
- Course tooling: Git/GitHub, Jira (or similar), Jenkins CI/CD, Docker, GoogleTest, SonarQube; agile development only.
- Six-phase semester timeline, so correctness of the core pipeline is prioritised over feature count.

3.2 Stakeholders & Concerns

Stakeholder	Concern	Architectural response
Project guide / evaluators	Clear, testable, explainable design; measurable results	Layered structure, per-block pipeline, statistics exposed by the core, traceability to SRS
Non-technical users	Simple compression and extraction with clear feedback	Web UI with progress, plain-language errors; all logic behind an HTTP API
Technical users	Scriptable, reproducible, integrity checks	CLI on the same core, deterministic archives, verify and info commands
Developers (team)	Parallel work, low coupling, easy to test	Module boundaries per owner, interfaces for Transform and EntropyCoder, core buildable without the UI
Maintainers	Extend without breaking old archives	Versioned format, stable transform IDs, transform registry

3.3 Component (UML) Diagram

Figure 1 shows the major components and their dependencies. Figure 2 shows the main classes and interfaces of the compression core.

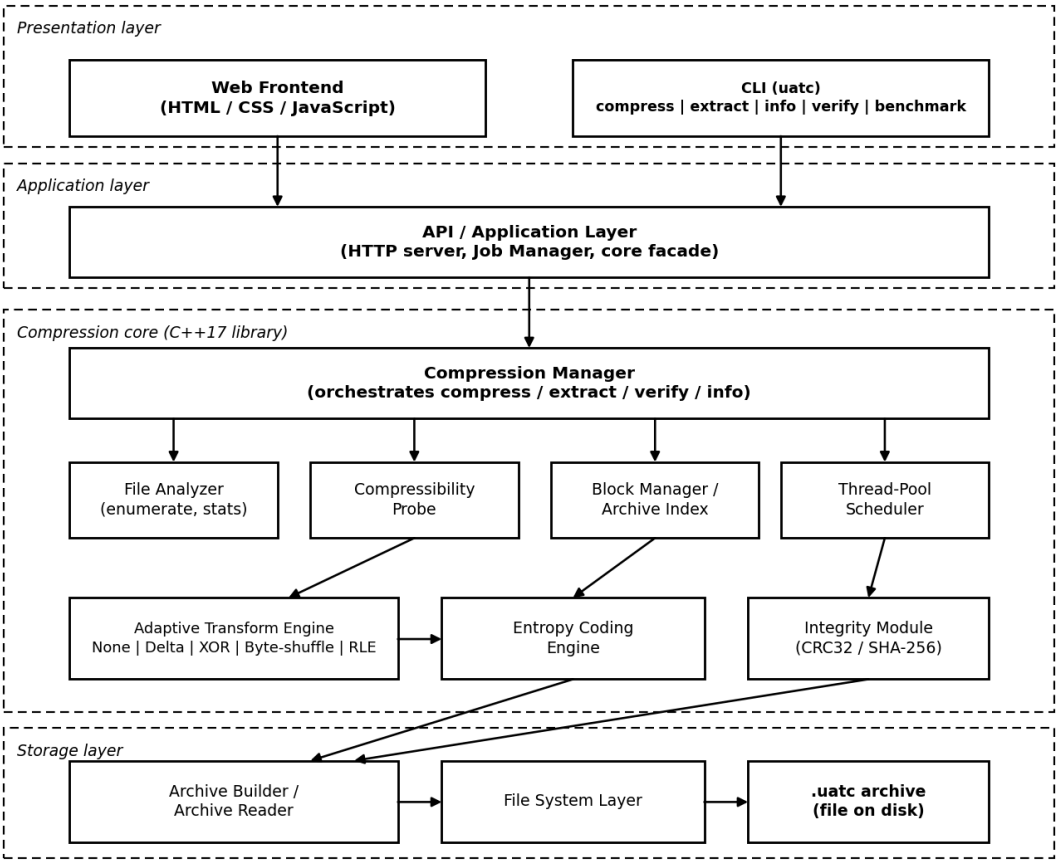
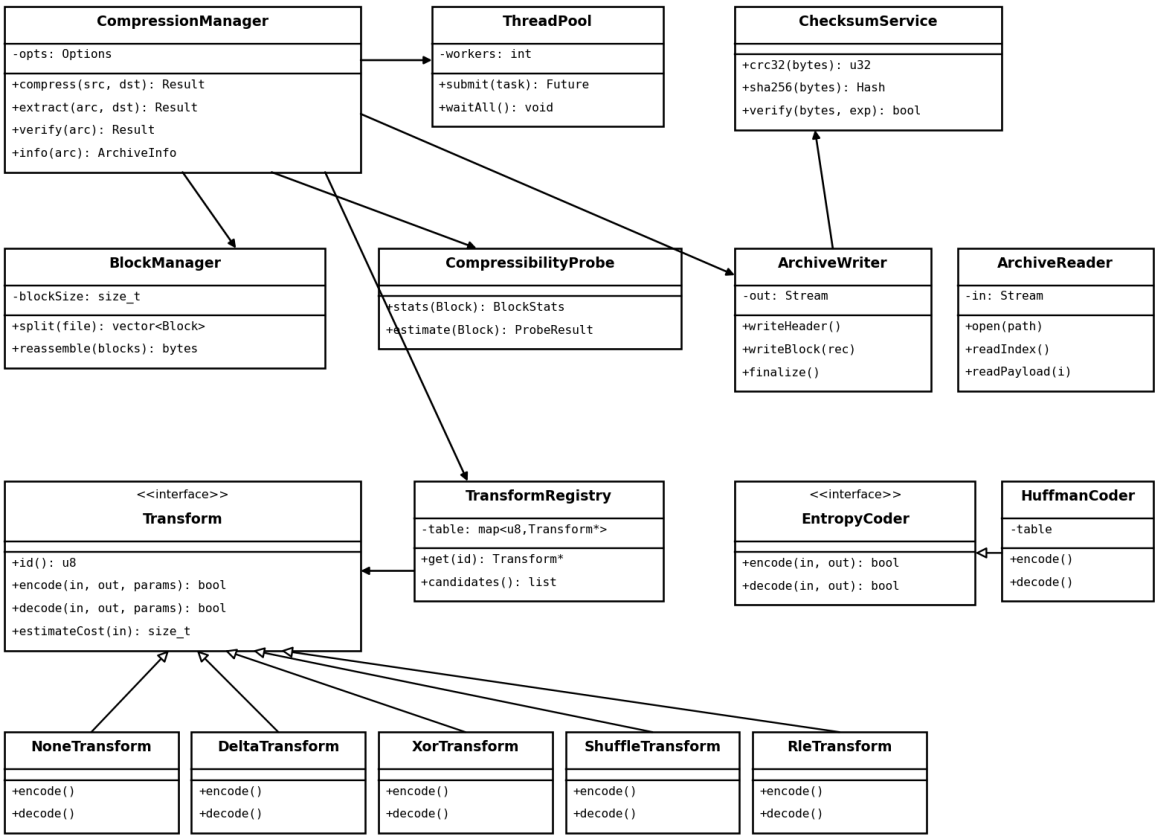


Figure 1: UATC component diagram



Solid arrow with open head = generalisation (implements); plain arrow = uses / depends on.

Figure 2: UML class diagram of the compression core

3.4 Component Descriptions

Component	Responsibility	Owner	SRS
Web Frontend	File/folder selection, Compress/Extract controls, progress, results and plain-language errors. No compression logic.	Manasa K	FR-13, FR-14
CLI	Sub-commands compress, extract, info, verify, benchmark; exit codes; calls the core facade directly.	All owners (own sub-commands); Manasa K integrates	FR-01 to FR-12
API / Application Layer	HTTP server, request validation, Job Manager (job table, status, progress), error mapping to JSON.	Manasa K	FR-15
Compression Manager	Orchestrates compress, extract, verify and info; owns the per-block pipeline order.	Sai Rishi	FR-02, FR-03
File Analyzer	Enumerates files/directories, collects metadata, computes per-block statistics.	S Chakshu / Sai Rishi	FR-01, FR-04
Compressibility Probe	Cheap estimate (e.g. entropy / repeat estimate on a sample) deciding if full compression is worth it.	Sai Rishi	FR-05
Block Manager / Archive Index	Splits files into blocks, tracks block records (offsets, lengths, IDs, checksums).	S Chakshu	FR-02
Adaptive Transform Engine	Common Transform interface, five v1 transforms, cost estimation and lowest-cost selection.	Sai Rishi	FR-06, FR-08
Entropy Coding Engine	Lossless entropy coding of the selected representation, and the decoder.	Sai Rishi	FR-07

Component	Responsibility	Owner	SRS
Thread-Pool Scheduler	Fixed worker pool; dispatches block tasks; returns results in block order.	Sampada K	FR-10
Integrity Module	CRC32 per block, SHA-256 per file/archive, verification and distinct error reporting.	S Chakshu	FR-09, FR-12
Archive Builder / Reader	Writes and reads the .uatc format; validates magic, version and structure.	S Chakshu	FR-02, FR-03, FR-12
Archive Inspector / Verifier	info and verify without extraction.	Manasa K	FR-11, FR-12
Benchmark Harness	Repeatable comparison against gzip/DEFLATE and Zstandard; timing and throughput instrumentation.	Sampada K	NFR Performance

3.5 Chosen Architecture Pattern and Rationale

UATC uses a **layered architecture** (presentation, application/API, compression core, storage) with a **pipeline** inside the core. Each block flows through probe, transform selection, entropy coding and checksum, and each stage sits behind an interface (Strategy pattern for transforms and entropy coders). Blocks do not depend on each other, so the pipeline is executed by a thread pool without changing the compression logic.

Layering keeps the C++ core buildable and testable without the web frontend, which is the main reason it was chosen. Microservices were rejected as unnecessary complexity for a single-machine tool; a dynamically loaded plugin system was rejected for the same reason, since a compile-time registry gives the same extensibility with less risk.

ADR	Decision	Alternatives rejected	Reason
ADR-01	Layered architecture with per-block pipeline	Microservices; single monolithic module	Simple, testable, matches team ownership split
ADR-02	Blocks are independently decodable; each has its own index record and CRC32	One stream per file	Enables parallelism and precise corruption reporting
ADR-03	C++ core exposed through an HTTP API to the JavaScript frontend	WebAssembly build; native GUI	Keeps core in C++17, GUI out of scope, simple integration
ADR-04	Transform selection by fast cost estimate, not full compression of every candidate	Try-all-and-measure	Keeps throughput benefit; selection stays deterministic
ADR-05	Results are written in block-index order; ties broken by lowest transform ID; no timing-based decisions	Write in completion order	Archive is identical for any worker count (FR-10)
ADR-06	Backend runs on the user's machine; files are chosen through a server-side file browser endpoint	Browser file upload	Browsers do not expose real paths; avoids copying large data through HTTP (see 4.6)
ADR-07	Progress via polling GET /api/jobs/{id}	WebSockets / SSE	Simplest to implement and test; enough for a 500 ms refresh

3.6 Technology Stack & Data Stores

Area	Choice	Note
Core language / build	C++17, CMake	Per SRS 2.5
Concurrency	std::thread based thread pool	Fixed worker count (--workers), serial path for small inputs

Area	Choice	Note
Entropy coding	Canonical Huffman coder behind an EntropyCoder interface	Standard technique; interface allows rANS or others later (final choice confirmed in Phase 2)
Checksums	CRC32 per block, SHA-256 per file and archive	SHA-256 also used for round-trip validation in integration tests
HTTP server / JSON	Lightweight embedded C++ HTTP library and JSON library (e.g. cpp-httplib, nlohmann/json)	Header-only, no external service; library choice recorded in Phase 2
Web frontend	HTML5, CSS3, vanilla JavaScript (fetch API)	No framework needed for four screens
Testing	GoogleTest, CTest, gcov/lcov for code and branch coverage, Selenium for UI	Coverage report is a course deliverable
CI/CD and quality	Git + GitHub, Jenkins, Docker, SonarQube, Jira	As required by the course guidelines
Data stores	The .uatc archive file is the only persistent store. Job state is held in memory by the Job Manager.	No database

3.7 Risks & Mitigations

Risk	Mitigation
A transform or coder bug corrupts data	Round-trip unit tests per transform, per-block CRC32, SHA-256 checks in integration tests, fuzzing
Parallel execution changes output (non-determinism)	Results ordered by block index (ADR-05); tests compare 1 vs N workers; ThreadSanitizer run in CI
Large inputs exhaust memory	Bounded block size and bounded queue of in-flight blocks; files streamed block by block
Malicious or damaged archive (path traversal, huge declared sizes)	Validate every index field before use; reject paths escaping the target directory; enforce size limits (see 3.9)
Long jobs freeze the web page	Jobs run asynchronously in the backend; UI only polls status
Frontend and backend drift apart	API contract fixed in 4.3 first; contract tests in CI
Scope too big for the semester	MVP first (Phase 2), stretch transform only after Phase 4
Parallelism overhead on small files	Serial path below a size threshold

3.8 Traceability to Requirements

Requirement	Component(s)	Design reference
FR-01 file/directory input	File Analyzer	4.1, 4.2
FR-02 archive creation	Archive Builder, Block Manager	4.1 (archive format)
FR-03 extraction	Archive Reader, Compression Manager	4.2 (Figure 4)
FR-04 block statistics	File Analyzer, Probe	4.1
FR-05 compressibility probe	Compressibility Probe	4.1
FR-06 transform selection	Transform Engine, Transform Registry	4.1, 4.3.2
FR-07 entropy coding	Entropy Coding Engine	4.3.2
FR-08 store decision	Transform Engine, Compression Manager	4.1
FR-09 checksum validation	Integrity Module	4.2 (Figure 4), 4.4
FR-10 parallel execution	Thread-Pool Scheduler	4.1, 4.2 (Figure 3)

Requirement	Component(s)	Design reference
FR-11 archive info	Archive Inspector	4.3.1
FR-12 error detection	Archive Reader, Integrity Module	4.3.4
FR-13 web interface	Web Frontend	4.5
FR-14 progress and status	Web Frontend, Job Manager	4.2 (Figure 3), 4.5
FR-15 frontend-backend interface	API / Application Layer	4.3.1
NFR Maintainability	Transform interface, Registry	4.3.2
NFR Observability	Compression Manager, Job Manager	4.4

3.9 Security Architecture

UATC handles untrusted input in two places: archives that are opened, and requests that reach the HTTP API. The table applies STRIDE to both.

Threat	Scenario	Mitigation
Spoofing	Another site or local process calls the HTTP API	Server binds to 127.0.0.1 only; strict CORS/Origin check; random per-session token required by the API
Tampering	Archive modified on disk or in transit	Per-block CRC32, archive/file SHA-256, header and index checksum; failures reported with block index
Repudiation	No record of what a job did	Job log with job ID, operation, sizes, result and error code (no file contents)
Information disclosure	API lists files outside the intended area; logs leak data	Path normalisation and allowed-root check on fs/list and job paths; no file contents or secrets in logs
Denial of service	Crafted archive declares huge sizes; very many tiny blocks	Limits on declared block length, block count and total output; bounded queues; reject before allocating
Elevation of privilege	Archive entry such as ../../x escapes the target folder (path traversal)	Reject absolute paths and .. segments; extraction writes only inside the chosen destination; never follows symlinks out of it

Additional controls: static analysis (SonarQube) and dependency review in CI; sanitizer builds (ASan/UBSan/TSan) for the test suite; fuzzing of the archive reader.

4. Design

4.1 Design Overview

UATC is designed as a set of small modules with fixed interfaces so that each team member can build and test one full functionality independently. The core is a static library (uatc_core) that is linked into the CLI and the HTTP server. The frontend contains only presentation code.

Repository layout

```
uatc/
  core/        analyzer, probe, block, transforms, entropy, scheduler, archive,
integrity
  cli/         uatc executable (argument parsing, output formatting, exit codes)
  server/      HTTP server, job manager, JSON mapping, error mapping
  web/         index.html, styles.css, app.js
  tests/       unit/, integration/, system/, fuzz/, data/
  bench/       benchmark harness
  docs/        SRS, SAD, STP, diagrams
  Jenkinsfile, Dockerfile, CMakeLists.txt
```

Per-block compression algorithm

For every block B (default size 1 MiB, configurable; the value is tuned in Phase 4 benchmarking):

- Compute lightweight statistics (byte histogram, entropy estimate, run count, delta statistics).
- Run the probe. If the block is estimated as incompressible, store it and stop.
- For each candidate transform Ti, estimate cost: metadata_cost(Ti) + encoded_size(Ti(B)) using a fast approximation. Choose T* with the lowest cost; ties go to the lowest transform ID.
- Apply T*, run the entropy coder, and add the block record (offset, lengths, transform ID, codec ID, parameters, CRC32).
- If the coded size plus metadata is not smaller than the original, store the original bytes instead (FR-08).

Extraction reverses this: read the record, entropy-decode, apply the inverse transform, compare CRC32, then write the bytes.

Archive format (ABI)

All multi-byte integers are little-endian. The block index has fixed-size entries, so its size is known before payloads are written; the writer reserves that region and fills in offsets once payloads are done.

Region	Contents
Magic + Version	4-byte magic "UATC", format version (major u16, minor u16). A reader rejects an unknown major version with E_UNSUPPORTED_VERSION.
Global Metadata	Flags, creation time, block-size policy, file count, block count, offsets of the following regions, header checksum
File Table	Per file: file ID, relative path (UTF-8, '/' separators), original size, timestamp/attributes, first block index, block count, SHA-256
Block Index	Per block: file ID, payload offset (u64), original length (u32), stored length (u32), transform ID (u8), codec ID (u8), transform parameters (u32), CRC32 (u32)
Block Payloads	Coded or raw bytes; each block decodable on its own
Integrity Data	SHA-256 over all preceding regions, followed by a footer with the header offset and magic

Transform IDs are stable and never reused: 0 = None (store), 1 = Delta, 2 = XOR, 3 = Byte-shuffle, 4 = RLE, 5 = reserved for bit-plane or the stretch-goal transform. Codec IDs: 0 = raw, 1 = Huffman.

Work division

Owner	Modules	Requirements
Sai Rishi Gangarapu (Lead)	Compression Manager, probe, transform engine, entropy coder, decoder	FR-04, FR-05, FR-06, FR-07
S Chakshu	File analyzer, Block Manager, Archive Builder/Reader, Integrity Module	FR-01, FR-02, FR-03, FR-09
Sampada K	Thread-Pool Scheduler, benchmark harness, performance instrumentation	FR-10, performance NFRs
Manasa K	Archive Inspector/Verifier, API layer, web frontend, integration	FR-11, FR-12, FR-13, FR-14, FR-15

4.2 UML Sequence Diagrams

Flow 1: compressing a folder from the web UI (FR-01, FR-02, FR-05 to FR-08, FR-10, FR-14, FR-15). The request returns a job ID immediately. The Compression Manager splits the input and submits block tasks to the thread pool. The UI polls the job status until it completes.

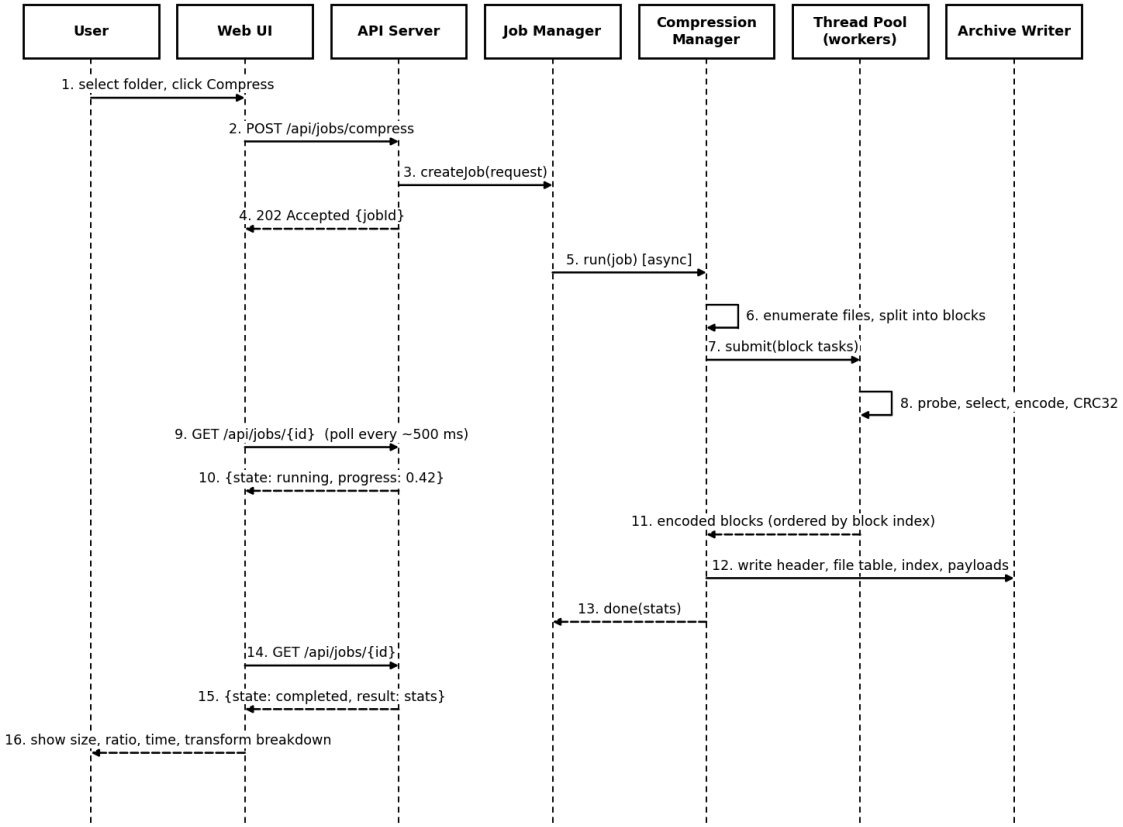


Figure 3: Sequence diagram – compress a folder via the web UI

Flow 2: extraction with integrity verification (FR-03, FR-09, FR-12). The reader validates magic and version first. Every block is decoded and its CRC32 checked before its bytes are written. A mismatch stops the job with a distinct error and removes partial output.

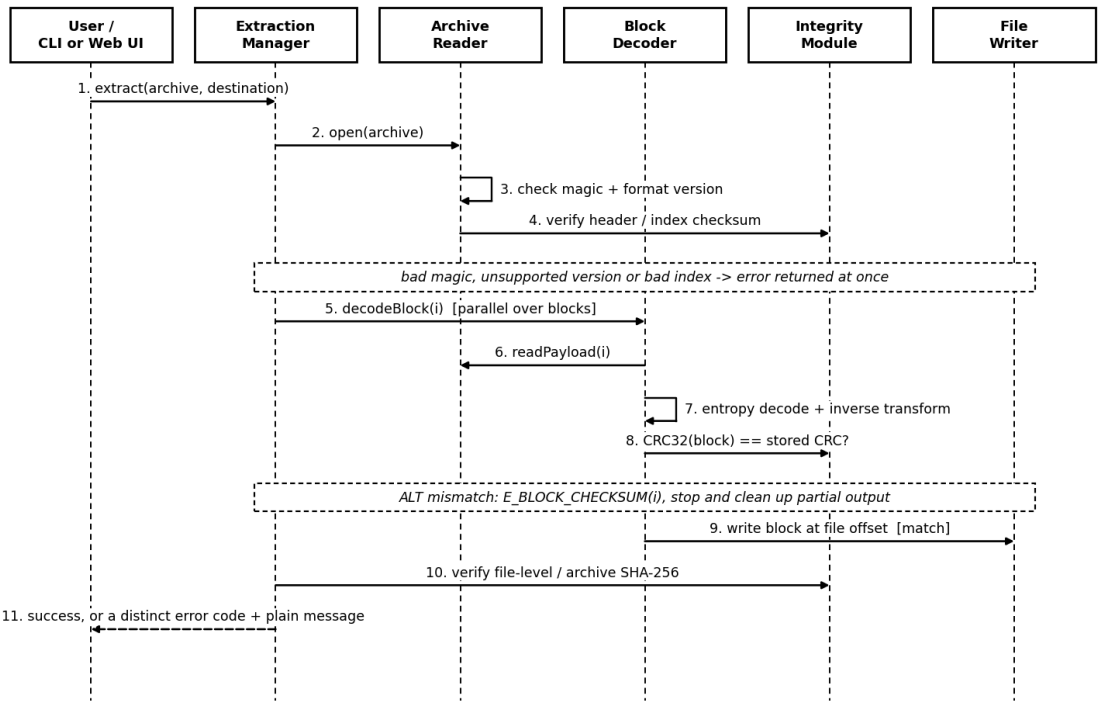


Figure 4: Sequence diagram – extract an archive with integrity checks

4.3 API Design

4.3.1 HTTP API (Web Frontend ↔ C++ backend)

Base path /api, JSON request and response bodies, UTF-8. The server listens on localhost only.

Method and path	Purpose	Success response
GET /api/health	Backend reachable check	200 {status:"ok", version}
GET /api/fs/list?path=	List a directory for the file picker	200 {path, entries:[{name, type, size}]}
POST /api/jobs/compress	Start a compression job	202 {jobId}
POST /api/jobs/extract	Start an extraction job	202 {jobId}
POST /api/jobs/verify	Verify an archive without extracting	202 {jobId}
GET /api/jobs/{id}	Job state, progress and result	200 {state, progress, result | error}
GET /api/archives/info?path=	Archive size, file count, block count, transform usage (FR-11)	200 {sizes, counts, transformUsage[]}

Example: start compression

```
POST /api/jobs/compress
Request : { "source": "/home/user/project", "destination": "/home/user/project.uatc",
            "workers": 8, "blockSize": 1048576 }        // workers, blockSize optional
Response: 202 { "jobId": "j-0007" }
Errors  : 400 E_INVALID_ARGUMENT, 403 E_PERMISSION, 404 E_NOT_FOUND
```

Example: job status

```
GET /api/jobs/j-0007
Response: 200 { "state": "completed",            // queued | running | completed |
failed
                "progress": 1.0,
                "result": { "originalBytes": 734003200, "archiveBytes": 412000000,
                            "ratio": 1.78, "elapsedMs": 5210, "throughputMBps": 134.5,
                            "blocks": 700, "stored": 41,
                            "transformUsage": [{"id":1,"name":"Delta","blocks":120},
...] } }
Failed job: 200 { "state": "failed", "error": { "code": "E_BLOCK_CHECKSUM",
                  "message": "...", "userMessage": "...", "details": {"block": 213} } }
Unknown ID: 404 E_NOT_FOUND
```

4.3.2 Transform and EntropyCoder interfaces (core, ABI between modules)

These two interfaces are what makes new transforms and coders pluggable. A new transform implements Transform and is added to the registry with a new, never-reused ID; nothing else changes.

```
class Transform {
 public:
  virtual ~Transform() = default;
  virtual uint8_t id() const = 0;                        // stable, never reused
  virtual size_t  estimateCost(ByteSpan in) const = 0;  // fast, deterministic
  virtual bool    encode(ByteSpan in, ByteBuffer& out, uint32_t& params) const = 0;
  virtual bool    decode(ByteSpan in, uint32_t params, ByteBuffer& out) const = 0;
};

class EntropyCoder {
 public:
  virtual ~EntropyCoder() = default;
  virtual uint8_t id() const = 0;
  virtual bool encode(ByteSpan in, ByteBuffer& out) const = 0;
  virtual bool decode(ByteSpan in, size_t origLen, ByteBuffer& out) const = 0;
};

// Contract: decode(encode(x)) == x for every input x, including empty input.
```

4.3.3 Core facade (used by CLI and API layer)

```
Result      compress(const Path& src, const Path& dst, const Options& opts,
ProgressSink*);
Result      extract (const Path& archive, const Path& dstDir, ProgressSink*);
Result      verify  (const Path& archive, ProgressSink*);
ArchiveInfo info    (const Path& archive);
// Options: workers, blockSize.  Result: ok flag, Stats, or Error{code, details}.
```

4.3.4 Error codes

Each failure case in FR-12 has its own code, so the CLI, API and UI can report them separately.

Code	Meaning	CLI exit	HTTP (direct call)	Plain-language message (UI)
E_INVALID_ARGUMENT	Bad option or path	2	400	Something in the request is not valid. Please check the file or folder you selected.
E_NOT_FOUND	Input or job not found	3	404	We could not find that file or folder.
E_PERMISSION	No permission / outside allowed root	4	403	UATC is not allowed to open or write that location.
E_IO / E_DISK_FULL	Read or write failure, disk full	5	500	The operation stopped because the disk could not be read or written (it may be full).
E_MALFORMED_ARCHIVE	Bad magic, broken structure, truncated file	10	422	This file is not a valid UATC archive or is damaged.
E_UNSUPPORTED_VERSION	Archive format version not supported	11	422	This archive was made by a newer version of UATC.
E_MISSING_BLOCK	Index refers to data that is not there	12	422	Part of this archive is missing.
E_BLOCK_CHECKSUM	Block CRC32 mismatch (block index reported)	13	422	This archive is corrupted and cannot be extracted safely.
E_ARCHIVE_CHECKSUM	Archive/file SHA-256 mismatch	14	422	This archive failed its integrity check.
E_PATH_TRAVERSAL	Archive path escapes destination	15	422	The archive contains an unsafe file path and was stopped.
E_INTERNAL	Unexpected error	1	500	Something went wrong inside UATC. Please try again.

4.4 Error Handling, Logging & Monitoring

- Errors are returned as values (Result / Error{code, message, details}), not thrown across module boundaries; the API layer maps them to JSON and the CLI to exit codes.

- The archive is written to a temporary file and renamed on success. On failure or cancellation, temporary and partial output created by the job is removed.
- Decoding never writes unchecked data: a block is written only after its CRC32 matches.
- Logging levels: ERROR, WARN, INFO, DEBUG. Logs contain job ID, operation, sizes, timings and error codes. They never contain file contents.
- Monitoring / observability (NFR): per job the core records original size, archive size, ratio, elapsed time, throughput, and the transform chosen for every block; counters for stored versus compressed blocks and checksum failures. GET /api/health supports the availability check in tests.
- CI monitoring: Jenkins build status, test pass rate, coverage and SonarQube quality gate are tracked per commit.

4.5 UX Design

Web UI (four screens).

- Home: two large choices, Compress and Extract.
- Select: file/folder browser backed by /api/fs/list, destination chooser, and a collapsed Advanced section (worker count).
- Progress: progress bar, percentage, elapsed time, and a status line. The page stays responsive while the job runs.
- Result: original size, archive size, ratio, time, throughput, and a small table of how many blocks used each transform. Errors use the plain-language messages from 4.3.4 and never show stack traces or raw codes.

Accessibility. Keyboard-operable controls, visible focus, labelled inputs, sufficient contrast, and status never conveyed by colour alone. Layout works from laptop to tablet widths.

CLI. Sub-commands compress, extract, info, verify, benchmark with --help for each; --workers N; --quiet and --verbose; progress on stderr and results on stdout so output can be scripted; non-zero exit codes as in 4.3.4.

4.6 Open Issues & Next Steps

- File selection method: a server-side file browser is used (ADR-06). If the guide prefers browser upload, an upload endpoint will be added; this only affects the API layer and frontend.
- Final entropy coder (Huffman vs rANS) and default block size: to be confirmed with Phase 2 and Phase 4 benchmark data.
- Whether bit-plane organisation is in the v1 candidate set or is the stretch-goal transform (ID 5 is reserved).
- Handling of symlinks, file permissions and timestamps across operating systems.
- Job cancellation and resume are not in v1; a cancel endpoint can be added without changing the archive format.
- Future enhancements: additional transforms, alternative entropy coders, distributed execution (all outside this semester).

5. Appendices

5.1 Glossary: UATC, block, transform, entropy coder, probe, store, CRC32, SHA-256, ADR, ABI, STRIDE, CLI (see Section 1.4).

5.2 References:

- UATC SRS v1.0 and Project Synopsis (Web Frontend, Final)
- IEEE 42010 (architecture description), IEEE 1016 (software design descriptions)
- RFC 1951 (DEFLATE) and RFC 8878 (Zstandard), used as comparison baselines
- OWASP guidance for path traversal and input validation; NIST SP 800-160

5.3 Tools: Git/GitHub, Jira, Jenkins, Docker, CMake, GoogleTest, gcov/lcov, SonarQube, Selenium; UML diagrams can be edited in draw.io or PlantUML.